

Projeto 2 - Expandindo Arrays Aninhados

video

05:22

Projeto 2 - Loops e Repetições

video

09:15

Projeto 2 - Pivot e Unpivot

video

06:14

Projeto 2 - Split Functions

video

07:14

Projeto 2 - Método Collect no Spark



ebook

01:39

Projeto 2 - Operadores



ebook

00:28

Projeto 2 - WithColumn



ebook

01:20

Projeto 2 - Operações de Colunas



ebook

01:32

Projeto 2 - MapType



ebook

02:01



Data Science
Academy

Operações de Colunas

As operações de colunas no Apache Spark são fundamentais para manipular DataFrames e realizar análises de dados complexas. Spark oferece uma ampla variedade de funções para trabalhar com colunas, permitindo desde operações simples, como adições e subtrações, até transformações mais avançadas, como aplicações de funções matemáticas, lógicas e de string.

Tipos de Operações de Colunas

1. Operações Aritméticas

Você pode realizar operações aritméticas básicas como adição, subtração, multiplicação e divisão diretamente nas colunas. Isso é útil para calcular novas métricas ou ajustar valores existentes.

```
1 from pyspark.sql import SparkSession
2 from pyspark.sql.functions import col
3
4 spark = SparkSession.builder.appName("Arithmetic Operations")
5 data = [(1, 100), (2, 200), (3, 300)]
6 df = spark.createDataFrame(data, ["id", "value"])
7
8 # Adicionando uma nova coluna que é o valor original multipl
9 df = df.withColumn("double_value", col("value") * 2)
10 df.show()
11
```

2. Operações Relacionais

Spark suporta operações relacionais para comparações.

```
1 from pyspark.sql.functions import when
2
3 # Criar uma nova coluna booleana baseada em condição
4 df = df.withColumn("is_large", when(col("value") > 150, True)
5 df.show()
6
```

3. Operações de String

Funções para manipulação de strings incluem **concat**, **substring**, **regexp_replace**, entre outras, que permitem manipular dados de texto.

```
1 from pyspark.sql.functions import concat, lit
2
3 # Concatenar strings em uma nova coluna
4 df = df.withColumn("description", concat(col("id"), lit(" -
5 df.show()
6
```

4. Transformações de Data e Hora

Spark fornece funções como **year**, **month**, **day**, **current_date** para trabalhar com datas, permitindo extrair partes ou calcular diferenças entre datas.

```
1 from pyspark.sql.functions import current_date, datediff
2
3 # Adicionar a data atual e calcular a diferença de dias
4 df = df.withColumn("today", current_date())
5 df = df.withColumn("days_from_today", datediff(col("today"),
6 df.show()
7
```

5. Funções Agregadas

Essas funções, como **sum**, **avg**, **max**, e **min**, são usadas para agregar dados, geralmente após um agrupamento.

```
1 df.groupBy().sum("value").show()
2
```

6. Funções Complexas

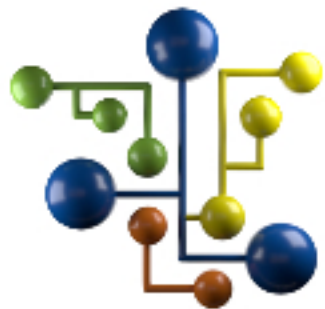
Spark também permite o uso de UDFs (User Defined Functions) e funções integradas avançadas como **explode**, que pode ser usada para expandir arrays ou mapas em linhas múltiplas.

```
1 from pyspark.sql.functions import explode
2
3 # Supondo que temos uma coluna 'array_col' com arrays
4 df = spark.createDataFrame([(1, [1, 2, 3]), (2, [4, 5])], ["
5 df.withColumn("exploded", explode(col("array_col"))).show()
6
```

Considerações de Desempenho

Enquanto essas operações oferecem grande flexibilidade e poder para manipular dados, elas também podem impactar o desempenho. Operações como **join**, **groupBy** e uso excessivo de UDFs podem ser particularmente custosas em termos de computação e uso de rede. É sempre uma boa prática monitorar o plano de execução de suas queries (usando **.explain()**) para entender e otimizar o processamento.

Usar eficientemente essas operações pode transformar a maneira como você acessa, transforma e analisa grandes conjuntos de dados em ambientes distribuídos usando Apache Spark.



Equipe DSA

Continue trilhando uma excelente
jornada de aprendizagem.

? Suporte

? Suporte

? Suporte

? Suporte

? Suporte

? Suporte

? Suporte

? Suporte

? Suporte

? Suporte